Que - 1: Fork() & Exec()

```
# Fork():-

#include<stdio.h>
#include<unistd.h>
#include<stdlib.h>
#include<sys/types.h>
void forkexample(){
    pid_t p = fork();
    if(p<0){
        perror("Fork failed");
        exit(1);
    }else if(p==0){
        printf("Hello from child");
    }else{
        printf("Hello from parent");
    }
}
int main(){
    forkexample();
    return 0;
}


# Exec():-

# ex1.c-
#include<stdio.h>
#include<unistd.h>
#include<stdlib.h>
int main(){
    printf("PID of ex1.c: %d\n", getpid());
    char *args[] = {"hello","everyone",NULL};
    execv("./ex2", args);
    perror("execv failed");
    printf("Back to ex1.c\n");
    return 0;
}
# ex2.c-
#include<stdio.h>
#include<unistd.h>
#include<stdlib.h>
int main(int args, char *argv[]){
    printf("We are in ex2.c\n");
    printf("PID of ex2.c: %d\n", getpid());
    for(int i=0; i<args; i++)
        printf("argv[%d]: %s\n", i, argv[i]);
    return 0;
}
```

Que - 2: Pipes (Parent & Child process)

```c
#include<stdio.h>
#include<unistd.h>
int main(){
    int pipefds[2];
    int returnstatus;
    int pid;
    char writemessages[2][20]={"Hi","Hello"};
    char readmessage[20];
    returnstatus = pipe(pipefds);
    if(returnstatus == -1){
        printf("Unable to create pipe\n");
        return 1;
    }
    pid = fork();
    if(pid==0){
        //Child Process
        read(pipefds[0],readmessage,sizeof(readmessage));
        printf("Child Process - Reading from pipe Message 1 is %s\n",readmessage);
        read(pipefds[0],readmessage,sizeof(readmessage));
        printf("Child Process - Reading from pipe Message 2 is %s\n",readmessage);
    }else{
        //Parent Process
        printf("Parent Process - Writing to pipe - Message 1 is
%s\n",writemessages[0]);
        write(pipefds[1],writemessages[0],sizeof(writemessages[0]));
        printf("Parent Process - Writing to pipe - Message 2 is
%s\n",writemessages[1]);
        write(pipefds[1],writemessages[1],sizeof(writemessages[1]));
    }
    return 0;
}
```

Que - 3: Message Passing

```
# Sender Program:-

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#define MSG_SIZE 100
struct message {
    long msg_type;
    char msg_text[MSG_SIZE];
};
int main() {
    key_t key;
    int msgid;
    struct message msg;
    key = ftok("progfile", 65);
    msgid = msgget(key, 0666 | IPC_CREAT);
    if (msgid == -1) {
        perror("msgget failed");
        exit(1);
    }
    msg.msg_type = 1;
    printf("Enter message to send: ");
    fgets(msg.msg_text, MSG_SIZE, stdin);
    msgsnd(msgid, &msg, sizeof(msg.msg_text), 0);
    printf("Sender: Message sent -> %s\n", msg.msg_text);
    return 0;
}


# Receiver Program:-

#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#define MSG_SIZE 100
struct message {
    long msg_type;
    char msg_text[MSG_SIZE];
};
int main() {
    key_t key;
    int msgid;
    struct message msg;
    key = ftok("progfile", 65);
    msgid = msgget(key, 0666 | IPC_CREAT);
    if (msgid == -1) {
        perror("msgget failed");
```

```
        exit(1);
    }
    msgrcv(msgid, &msg, sizeof(msg.msg_text), 1, 0);
    printf("Receiver: Message Received -> %s\n", msg.msg_text);
    msgctl(msgid, IPC_RMID, NULL);
    return 0;
}
```

Que - 4: Shared Memory

```
# Writer Program:-
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#define SHM_SIZE 1024
int main(){
    key_t key;
    int shmid;
    char *shared_mem;
    key = ftok("progfile", 75);
    shmid = shmget(key, SHM_SIZE, 0666 | IPC_CREAT);
    if (shmid == -1){
        perror("shmget failed");
        exit(1);
    }
    shared_mem = (char *)shmat(shmid, NULL, 0);
    printf("Enter data to write to shared memory: ");
    fgets(shared_mem, SHM_SIZE, stdin);
    printf("Writer: Data written -> %s\n", shared_mem);
    shmdt(shared_mem);
    return 0;
}


# Reader Program:-
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#define SHM_SIZE 1024
int main(){
    key_t key;
    int shmid;
    char *shared_mem;
    key = ftok("progfile", 75);
    shmid = shmget(key, SHM_SIZE, 0666 | IPC_CREAT);
    if (shmid == -1){
        perror("shmget failed");
        exit(1);
    }
    shared_mem = (char *)shmat(shmid, NULL, 0);
    printf("Reader: Data read -> %s\n", shared_mem);
    shmdt(shared_mem);
    shmctl(shmid,IPC_RMID,NULL);
    return 0;
}
```

Que - 5: FCFS

```c
#include <stdio.h>
int main(){
    int n,sum,i,j;
    printf("Enter the total no. of Processes:");
    scanf("%d",&n);
    int at[n],bt[n],wt[n],tat[n],ct[n];
    float totalwt=0,totaltat=0;
    printf("\nEnter the process arrival time and Burst Time:\n");
    for (i=0;i<n;i++){
        printf("Enter arrival time of Process[%d]: ", i + 1);
        scanf("%d",&at[i]);
        printf("Enter burst time of Process[%d]: ", i + 1);
        scanf("%d",&bt[i]);
    }
    sum=at[0];
    for(j=0;j<n;j++){
        sum+=bt[j];
        ct[j]=sum;
        tat[j]=ct[j]-at[j];
        wt[j]=tat[j]-bt[j];
        totaltat+=tat[j];
        totalwt+=wt[j];
    }
    printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
    for(i=0;i<n;i++){
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",i+1,at[i],bt[i],ct[i],tat[i],wt[i]);
    }
    printf("\nAverage Turn Around Time = %.1f", totaltat/n);
    printf("\nAverage Waiting Time = %.1f", totalwt/n);
}
```

Que - 6: SJF

```c
#include <stdio.h>
int main(){
    int n, sum, i, j, temp;
    float totaltat = 0, totalwt = 0;
    printf("Enter total No of processes:");
    scanf("%d", &n);
    int bt[n], ct[n], wt[n], tat[n], p[n];
    printf("\nEnter process number:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &p[i]);
    printf("\nEnter Process Burst Time\n");
    for(i = 0; i < n; i++){
        // p[i] = i + 1;
        printf("Enter Burst time of process[%d]:", i + 1);
        scanf("%d", &bt[i]);
    }
    /* Apply Bubble sort to sort */
    for(i = 0; i < n; i++){
        for(j = 0; j < n - 1 - i; j++){
            if(bt[j] > bt[j + 1]){
                temp = bt[j];
                bt[j] = bt[j + 1];
                bt[j + 1] = temp;
                temp = p[j];
                p[j] = p[j + 1];
                p[j + 1] = temp;
            }
        }
    }
        sum = 0;
    for(j = 0; j < n; j++){
        sum = sum + bt[j];
        ct[j] = sum;
        tat[j] = ct[j];
        totaltat = totaltat + tat[j];
        wt[j] = tat[j] - bt[j];
        totalwt = totalwt + wt[j];
    }
    printf("\nProcess\tBT\tTAT\tWT\n\n");
    for(i = 0; i < n; i++){
        printf("\nP%d\t %d\t %d\t %d\t\n",i+1, bt[i], tat[i], wt[i]);
    }
    printf("\nAvg Turnaround Time:%f\n",totaltat/n);
    printf("\nAvg Waiting Time:%f",totalwt/n);
    return 0;
}
```

Que - 7: Semaphores

```c
#include<stdio.h>
#include<unistd.h>
int mutex=1,full=0,empty=3,x=0;
int main(){
    int n;
    void producer();
    void consumer();
    int wait(int);
    int signal(int);
    printf("\n1.Producer\n2.Consumer\n3.Exit");
    while(1){
        printf("\nEnter your choice:");
        scanf("%d",&n);
        switch(n){
            case 1: if((mutex==1) && (empty!=0))
                        producer();
                    else
                        printf("\nBuffer is full");
                    break;
            case 2:if((mutex==1) && (full!=0))
                        consumer();
                    else
                        printf("\nBuffer is empty");
                    break;
            case 3:exit(0);
                    break;
        }
    }
    return 0;
}
int wait(int s){ return(--s); }
int signal(int s){ return(++s); }
void producer(){
    mutex=wait(mutex);
    full=signal(full);
    empty=wait(empty);
    x++;
    printf("\nProducer produces the item %d",x);
    mutex=signal(mutex);
}
void consumer(){
    mutex=wait(mutex);
    full=wait(full);
    empty=signal(empty);
    printf("\nConsumer consumes the item %d",x);
    x--;
    mutex=signal(mutex);
}
```

Que - 8: MVT

```c
#include <stdio.h>
#include <conio.h>
int main(){
    int i, m, n, tot, s[20];
    printf("Enter total memory size:");
    scanf("%d", &tot);
    printf("Enter no. of processes:");
    scanf("%d", &n);
    printf("Enter memory for OS:");
    scanf("%d", &m);
    for(i = 0; i < n; i++){
        printf("Enter size of process %d:",i+1);
        scanf("%d", &s[i]);
    }
    tot = tot - m;
    for(i = 0; i < n; i++){
        if(tot >= s[i]){
            printf("Allocate page %d\n",i+1);
            tot=tot-s[i];
        }else
            printf("process P%d is blocked\n",i+1);
    }
    printf("External fragmentation is = %d",tot);
    return 0;
}
```

Que - 9: First Fit

```c
#include <stdio.h>
#define max 25
int main(){
    int frag[max], b[max], f[max];
    int i, j, nb, nf, temp;
    static int bf[max], ff[max];
    printf("Enter the number of blocks: ");
    scanf("%d", &nb);
    printf("Enter the number of files: ");
    scanf("%d", &nf);
    printf("\nEnter the size of the blocks:\n");
    for(i = 1; i <= nb; i++){
        printf("Block %d: ", i);
        scanf("%d", &b[i]);
    }
    printf("\nEnter the size of the files:\n");
    for(i = 1; i <= nf; i++){
        printf("File %d: ", i);
        scanf("%d", &f[i]);
    }
    for(i = 1; i <= nf; i++){
        ff[i] = 0;    // Initially no block is allocated
        for(j = 1; j <= nb; j++){
            if(bf[j] != 1){
                temp = b[j] - f[i];
                if(temp >= 0){
                    ff[i] = j;
                    frag[i] = temp;
                    bf[j] = 1;
                    break;
                }
            }
        }
    }
    printf("\nFile_no\tFile_size\tBlock_no\tBlock_size\tFragment");
    for(i = 1; i <= nf; i++){
        if(ff[i] != 0){
            printf("\n%d\t%d\t\t%d\t\t%d\t\t%d",i, f[i], ff[i], b[ff[i]], frag[i]);
        }else
            printf("\n%d\t%d\tNot Allocated\t-\t\t-",i, f[i]);
    }
    return 0;
}
```

Que - 10: FIFO

```c
#include <stdio.h>
int main(){
    int frames[10],ref[50];
    int nof,nor;
    int i,j,pageFault=0,found,pointer=0;
    printf("FIFO PAGE REPLACEMENT ALGORITHM\n");
    printf("Enter no of frames: ");
    scanf("%d",&nof);
    printf("Enter no of reference strings: ");
    scanf("%d",&nor);
    printf("Enter Reference String: ");
    for(i = 0; i < nor; i++)
        scanf("%d", &ref[i]);
    for(i = 0; i < nof; i++)
        frames[i] = -1;
    printf("\nReference\tFrames\t\tStatus\n");
    for(i=0;i<nor;i++){
        found=0;
        for(j=0;j<nof;j++){
            if (frames[j]==ref[i]){
                found=1;
                break;
            }
        }
        if(found==0){
            pageFault++;
            frames[pointer]=ref[i];
            pointer=(pointer+1)%nof;
        }
        printf("%d\t\t", ref[i]);
        for (j = 0; j < nof; j++){
            if (frames[j] == -1)
                printf("- ");
            else
                printf("%d ", frames[j]);
        }
        if (found == 0)
            printf("\t\tPage Fault");
        else
            printf("\t\tPage Hit");
        printf("\n");
    }
    printf("\nNo of page faults: %d\n", pageFault);
    return 0;
}
```